<딥러닝 기초>

참고자료1. Jupyter Notebook

목차

- (1) IPython과 Jupyter
  - IPython (Interactive Python)
  - Project Jupyter
  - Jupyter Notebook
  - JupyterLab
- (2) Anaconda 설치
  - Anaconda
  - Conda
  - Anaconda 다운로드 및 설치
  - Anaconda 설치 확인
    - Windows
    - Linux
    - Anaconda 가상 환경
- (3) Jupyter Notebook 실행해 보기
  - Localhost (설치 컴퓨터) 실행
  - Juyter Notebook 비밀번호 설정
- (4) Jupyter Notebook의 기본 기능
  - Notebook Dashboard
  - Notebook Edior
  - 새 노트북 생성
  - Editor Cell 유형
  - 자동완성 기능
  - 객체정보 조회
  - Cell 실행
  - Cell 출력 결과 활용
  - 참고자료
- (5) Markdown 사용법
  - 제목
  - 리스트
  - 프로그램 코드
  - 수식 (Latex 형식)
  - 테이블
  - 수평선
  - 파일
  - 그림 파일
  - 텍스트 강조
  - 참고
- (6) Jupyter Notebook의 숨은 기능
  - Magic Commnads
  - %%writefile를 이용해서 현재 위치에 add.py 생성하고 조회(%ls)
  - %loadpy를 이용해서 python 파일을 로드 및 실행
  - %timeit 를 사용해서 실행하는 시간을 점검
  - Shell 명령

(1) IPython과 Jupyter

IPython (Interactive Python)

- 2013년 Fernando Perez가 개발
- 여러 프로그래밍 언어의 대화형 컴퓨팅을위한 명령 셸로, 원래 Python 프로그래밍 언어 용으로 개발되었으며 introspection, rich media, 셸 구문, 탭 완성 및 히스토리 기록 등 제공

Project Jupyter

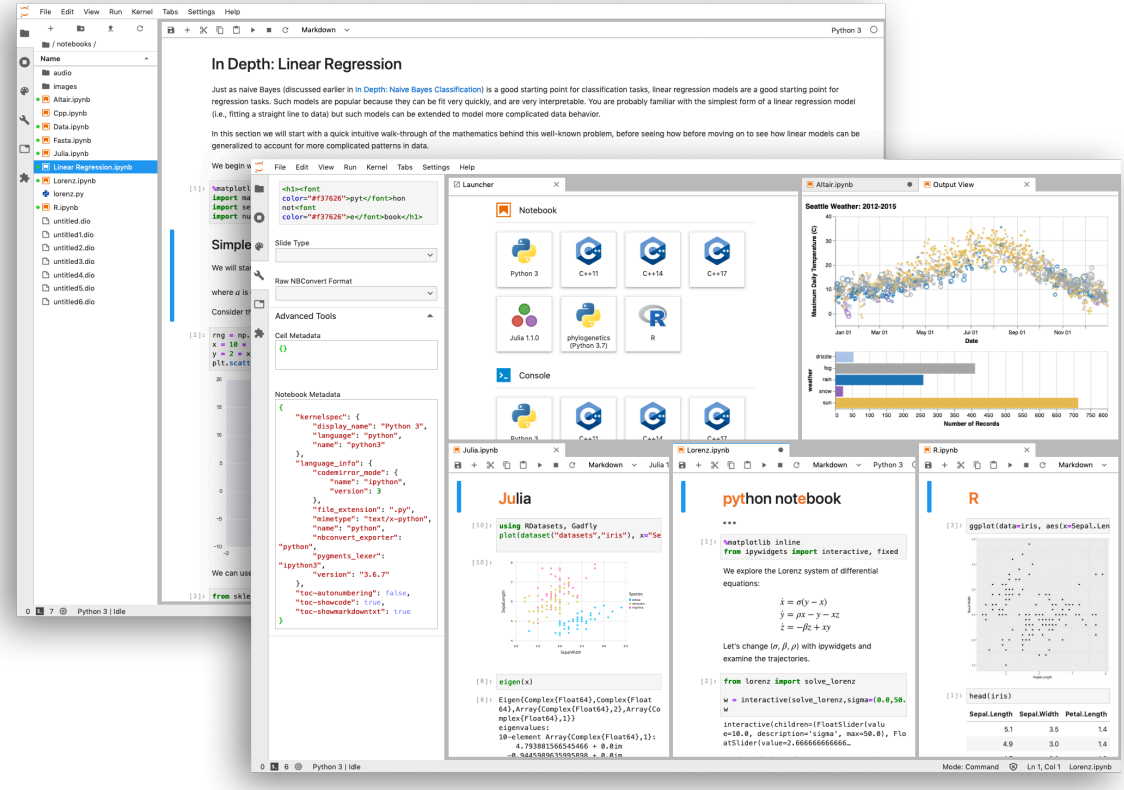
- 2014 년에 시작된 IPython의 스핀오프 프로젝트
- IPython은 계속해서 Jupyter 용 Python 셸 및 커널로 존재
- IPython의 노트북 인터페이스 및 기타 언어에 구애받지 않는 부분이 Jupyter 이름으로 이동
- Jupyter는 Julia, Python 및 R 등의 언어 지원

Jupyter Notebook

- 라이브 코드, 방정식, 시각화 및 설명 텍스트가 포함 된 문서를 만들고 공유 할 수있는 오픈 소스 웹 애플리케이션
- 데이터 정리 및 변환, 수치 시뮬레이션, 통계 모델링, 데이터 시각화, 기계 학습 등에 사용

JupyterLab

- Jupyter의 차세대 노트북 인터페이스
- Jupyter 노트북, 코드 및 데이터를 위한 웹 기반 대화 형 개발 환경
- 유연성 - 데이터 과학, 과학 컴퓨팅 및 기계 학습에서 다양한 워크 플로우를 지원하도록 사용자 인터페이스를 구성하고 정렬
- 확장성 및 모듈 기반 - 새 구성 요소를 추가하고 기존 구성 요소와 통합하는 플러그인 작성 가능



---

(2) Anaconda 설치

Anaconda

- 과학 컴퓨팅 (데이터 과학, 기계 학습 애플리케이션, 대규모 데이터 처리, 예측 분석 등)을 위한 Python 및 R 프로그래밍 언어의 무료 오픈 소스 배포판
- 250 개가 넘는 패키지가 자동으로 설치되며, 7,500 개 이상의 추가 오픈 소스 패키지를 PyPI와 conda 패키지 및 가상 환경 관리자에서 설치할 수 있음

Conda

- 오픈 소스, 크로스 플랫폼, 언어에 구애받지 않는 패키지 관리자 및 환경 관리 시스템
- 패키지 및 해당 종속성을 설치, 실행 및 업데이트함

Anaconda 다운로드 및 설치

다운로드 페이지

- https://www.anaconda.com/products/individual

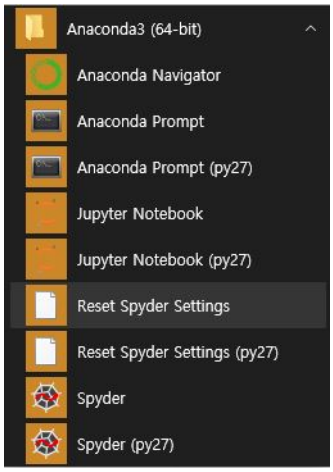
OS에 따라 다운로드 및 설치

- linux: `$ bash Anaconda3-xxx-Linux-x86_64.sh`

Anaconda 설치 확인

Windows

- 앱 목록에 Anaconda3 (64-bit) 생성



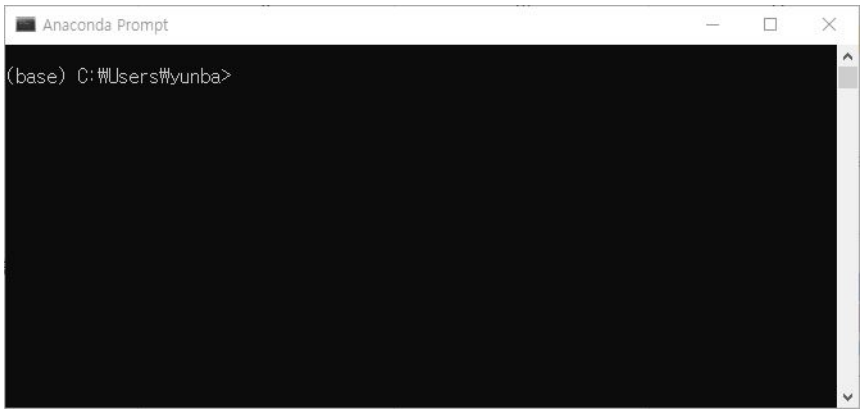
- Anaconda Navigator: GUI 기반으로 설치된 응용 패키지를 관리
- Anaconda Prompt: Anaconda 가상환경 (base)으로 터미널 실행
- Jupyter Notebook: Jupyter Notebook 바로 실행 (기본 http://127.0.0.1:8888)
- Spyder: 과학/공학 개발자를 위한 Python IDE

Linux

- 터미널을 통해 Anaconda base 가상환경 확인

Anaconda 가상 환경

- 다른 버전의 Python 및 / 또는 패키지 설치 환경을 create, export, list, remove, update 가능.
- 환경 활성화(activate)를 통해 환경 간 전환 또는 이동



---

(3) Jupyter Notebook 실행해 보기

Localhost (설치 컴퓨터) 실행

- Anaconda 가상환경에서 jupyter notebook 실행

`base) C:\Users\Name> jupyter notebook`

`base) hostname@account:~$ jupyter notebook`

- 기본 실행 URL: http://127.0.0.1:8888
- 처음 실행시 기본 포트: 8888, 추가 실행 시 포트 번호 1씩 증가

Juyter Notebook 비밀번호 설정

`$ jupyter notebook password`

Juyter Notebook 외부 접속 설정

- Juyter Notebook 설정 파일 생성 (jupyter_notebook_config.py)

`$ jupyter notebook --generate-config`

- Juyter Notebook 설정 파일 편집 (jupyter_notebook_config.py)

- 다음 내용 수정

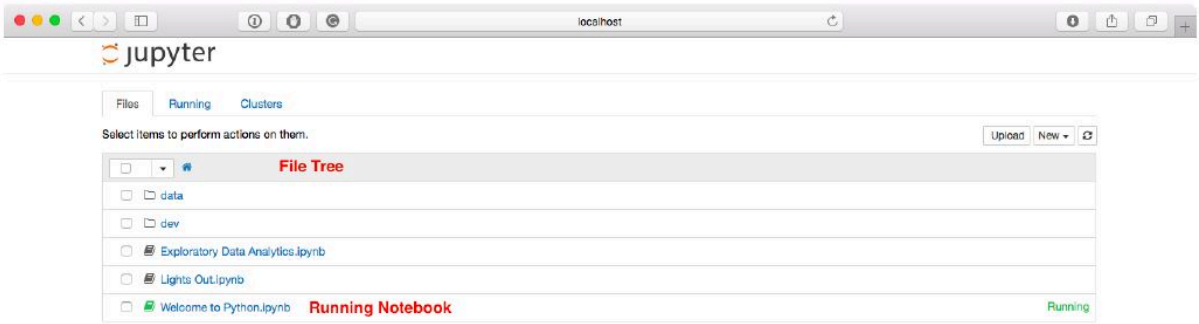
`#c.NotebookApp.allow_remote_access = False` ⇒ `c.NotebookApp.allow_remote_access = True`

`#c.NotebookApp.ip = 'localhost'` ⇒ `c.NotebookApp.ip = '202.31.xxx.yyy'` (localhost의 IP 주소)

---

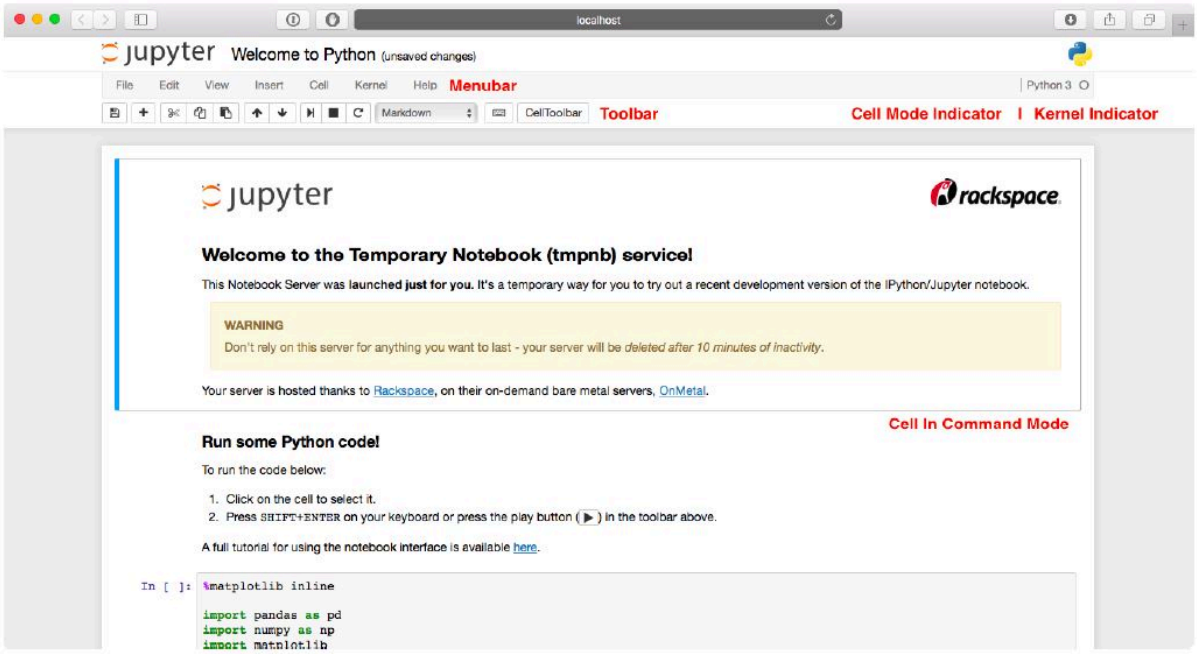
(4) Jupyter Notebook의 기본 기능

Notebook Dashboard



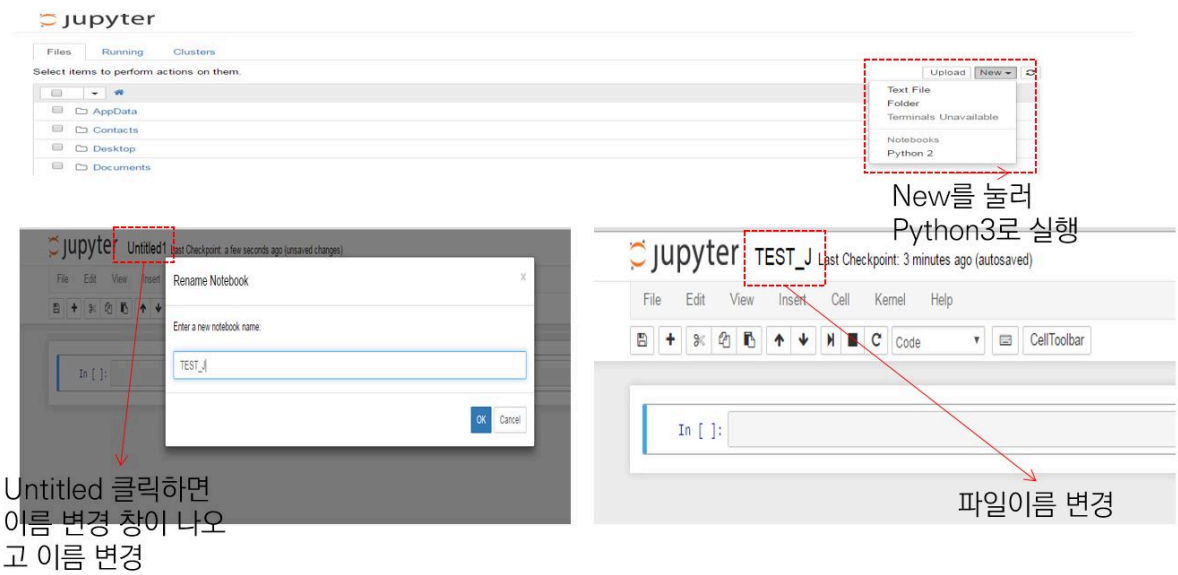
Notebook Edior

- Dashboard의 파일 리스트에서 노트북 파일(확장자: ipynb)을 선택하거나 New 메뉴에서 커널(Python3 등)을 선택(새 노트북 생성)하면 Notebook Editor로 전환



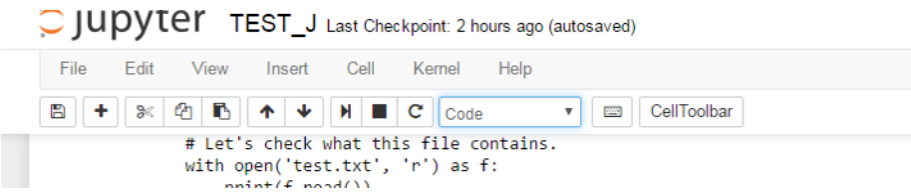
새 노트북 생성

- Dashboard에서 커널을 선택하여 실행
- 노트북 파일 기본 이름 변경

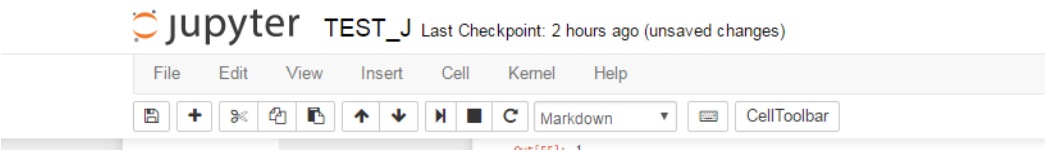


Editor Cell 유형

- Code Cell : 파이썬 등 코드 입력 및 실행

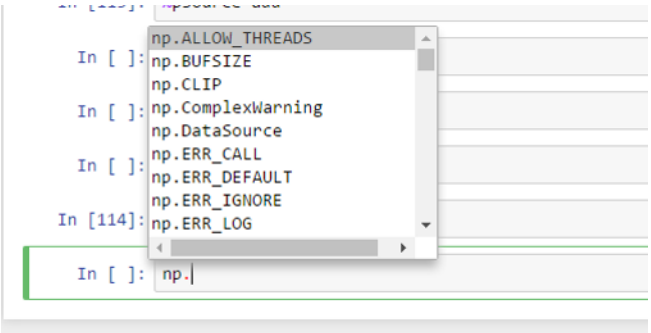


- Markdown Cell : markdown 형식으로 문서 작성



자동완성 기능

- 모듈(클래스) 이름을 입력하고 tab 키를 누르면 내부에 있는 요소들을 보여주므로 선택해서 사용 가능




```
In [1]: import numpy as np
```

```
In [2]: np.
```

```
  Input In [2]
    np.
       ^
SyntaxError: invalid syntax
```

객체정보 조회

- 변수 또는 함수 입력시 shift+TAB을 누르면 내부 특성이 조회 됨

```
import numpy as np

x = [1, 2, 3, 99, 99, 3, 2, 1]
x
```

```
Type:        list
String form: [1, 2, 3, 99, 99, 3, 2, 1]
Length:      8
Docstring:
```

```
def add(x:int, y:int) ->int :
    return x+y

add
```

```
Signature: add(x:int, y:int) -> int
Docstring: <no docstring>
File:      c:\users\06411\documents\<ipython-input-820-5bfdc7793909>
Type:      function
```

In [3]:
```
x = [1,2,3]
```

In [4]:
```
x
```

Out[4]:
```
[1, 2, 3]
```

Cell 실행

- Shift + Enter: 실행 후 다음 셀 활성화
- Ctrl + Enter: 실행 후 다음 셀 비활성화

In [5]:
```
# 객체 입력후 ?를 붙이고 실행(Shift-Enter)시키면 내부 정보가 보임
list?
```

In [6]:
```
# 객체 입력후 ??를 이용하면 help 정보(소스)가 나옴
sum??
```

Cell 출력 결과 활용

- _ 는 앞의 출력 결과를 가져오는 데 사용

In [7]:
```
1+1
```

Out[7]:
```
2
```

In [8]:
```
_ + 10
```

Out[8]:
```
12
```

참고자료

- 온라인 매뉴얼: https://jupyter-notebook.readthedocs.io/en/stable/notebook.html
- PDF 매뉴얼: pdf_manual
- Notebook으로 작성된 매뉴얼: https://jupyter.brynmawr.edu/services/public/dblank/Jupyter%20Notebook%20Users%20Manual.ipynb

---

(5) Markdown 사용법

제목

- 코드

  ```
  # Heading 1
  ## Heading 2
  ### Heading 3
  #### Heading 4
  ##### Heading 5
  ```
- 결과

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

리스트

- 코드

```
* This is a bullet list
  - sub item
  - sub item
     * sub sub item
* This is a bullet list
* This is a bullet list
1. And you can also create ordered lists

  1.1

     1.1.1

     1.1.2

  1.2

2. by using numbers
3. and listing new items in the lists
4. on their own lines
```

- 결과

  - This is a bullet list
    - sub item
    - sub item
      - sub sub item
  - This is a bullet list
  - This is a bullet list

  1. And you can also create ordered lists

     1.1 test

     1.1.1 test

     1.1.2 test

     1.2 test

  2. by using numbers

  3. and listing new items in the lists

  4. on their own lines

프로그램 코드

- in-line 코드

  This is an `in-line` code block.

- 일반 코드 블록

```
def f(x):
    """a docstring"""
    return x**2
```

- Highlighted 코드 블록

```
def f(x):
    """a docstring"""
    return x**2
```

수식 (Latex 형식)

- in-line 수식

```
This is: $e^{i\pi} + 1 = 0$.
```

This is: $e^{i\pi} + 1 = 0$.

- 행 전체 수식

```
$$e^x=\sum_{i=0}^\infty \frac{1}{i!}x^i$$
```

$$e^x=\sum_{i=0}^\infty \frac{1}{i!}x^i$$

테이블

- 코드

```
| Default aligned | Left aligned | Center aligned  | Right aligned  |
|-----------------|:-------------|:---------------:|---------------:|
| First body part | Second cell  | Third cell      | fourth cell    |
| Second line     | foo          | **strong**      | baz            |
| Third line      | quux         | baz             | bar            |
```

- 결과

Default aligned	Left aligned	Center aligned	Right aligned
First body part	Second cell	Third cell	fourth cell
Second line	foo	**strong**	baz
Third line	quux	baz	bar

수평선

- 코드

```
---
***
```

- 결과

파일

- 코드

```
[filename](jupyter-notebook-readthedocs-io-en-5.3.1.pdf)
```

- 결과

filename

그림 파일

- 코드

```
![title](img/nplab.PNG)
```

- 결과

- 코드

```
<img src="img/nplab.png" width="300" height="300">
```

- 결과

텍스트 강조

- 코드

```
This is **bold** and this is _italic_.
```

- 결과

  This is **bold** and this is *italic*.

참고

- Markdown Guide: https://about.gitlab.com/handbook/markdown-guide/
- Markdown Cell: https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Working%20With%20Markdown%20Cells.html

---

(6) Jupyter Notebook의 숨은 기능

Magic Commnads

명령어	설명
%pwd, %cd	**현재 위치 및 다른 디렉토리로 이동**
%history	**명령어 히스토리 출력**
%reset	**모든 정의된 변수 삭제**
%%capture	실행되는 명령에 대한 정보의 결과를 저장
%whos	**현재 정의된 변수 표시**
%pdoc, %psource	Help 기능 실행
%timeit	**평균 실행 시간을 출력**
%bookmark	디렉토리에 대한 별칭을 저장하고 쉽게 이동할 수 있게 해줌
%%writefile	**현재 디렉토리에 파일 생성**
%load	디렉토리에 있는 파일을 셀에 로딩
%run	py 프록램 파일을 실행
%matplotlib inline	matplotlib을 내부 셀에서 실행하기
%ls	**현재 디렉토리에 파일들을 보기**
%magic	모든 매직 함수에 대한 상세 도움말 출력
%pdb	예외가 발생하면 자동적으로 디버거 진입.(한번 입력시 ON, 다시 입력시 OFF)
%debug	작성된 코드에 대한 debug 처리

- 맨 첫줄에 사용해야 함

%%writefile를 이용해서 현재 위치에 add.py 생성하고 조회(%ls)

In [9]:
```
%%writefile add.py

def add(x,y):
    return x+y

print(add(4,4))
```

```
Overwriting add.py
```

In [10]:
```
%ls
```

0강. 딥러닝 과목 소개 (2020).pptx
0강. 딥러닝 과목 소개 (2021).pptx
0강. 딥러닝 과목 소개 (2022).pptx
Backup/
Class01-JupyterNotebook.ipynb
Class02-Numpy.ipynb
Class03-Math-선형대수.ipynb
Class04-Math-다변량미적분.ipynb
Class05-Math-확률통계.ipynb
Class06-머신러닝 기초.ipynb
Class07-인공신경망 이해하기.ipynb
Class08-다층 인공신경망 구성하기.ipynb
Class09-인공신경망 학습 이해하기.ipynb
Class10-인공신경망 학습 구현하기.ipynb
Class11-인공신경망 학습 개선하기.ipynb
Class12-최적화 알고리즘 구현하기.ipynb
Class13-과적합방지 알고리즘 구현하기.ipynb
Class14-가중치 초기값 설정 및 하이퍼파라미터 값 찾기.ipynb
Class14-가중치 초기값 설정 및 하이퍼파라미터 값 찾기.zip
Class15-인공신경망 학습 코드 관리하기.ipynb
Class16-GPU 연산과 딥러닝 플랫폼.ipynb
Class16-GPU 연산과 딥러닝 플랫폼.zip
Class17-공용 실습 환경 활용하기.ipynb
Class18-Tensorflow 및 Keras 이해하기.ipynb
Class18-Tensorflow 및 Keras 이해하기.zip
Class19-PyTorch 이해하기.ipynb
Class19-PyTorch 이해하기.zip
Class20-CNN의 이해와 구현.ipynb
Class20-CNN의 이해와 구현.zip
Class21-CNN 처음부터 구현해보기.ipynb
Class21-CNN 처음부터 구현해보기.zip
Class22-CNN 모델 활용하기.ipynb
Class22-CNN 모델 활용하기.zip
Class23-CNN 시각화 및 데이터 증가 활용하기.zip
Class23-CNN 시각화 및 데이터 증강 활용하기.ipynb
Class23-CNN 시각화 및 데이터 증강 활용하기.zip
Class24-딥러닝 디자인 패턴.ipynb
Class25-CNN 기반 객체 탐지 모델 만들기.zip
Class25-CNN 기반 객체 탐지의 이해.ipynb
Class25-CNN 기반 객체 탐지의 이해.zip
Class26-CNN 기반 객체 탐지 모델 만들기-수정.ipynb
Class26-CNN 기반 객체 탐지 모델 만들기.ipynb
Class26-딥러닝 디자인 패턴.zip
Class27-RNN의 이해와 구현.ipynb
Class27-RNN의 이해와 구현.zip
Class27.RNN의 이해와 구현 (4).mp4
Class28-RNN 기반 자연어 처리.ipynb
Class28-RNN 기반 자연어 처리.zip
Class29-Transformer 기반 자연어 처리(TBD).ipynb
Class30-시계열 데이터 예측하기.ipynb
Class31-GAN의 이해와 구현.ipynb
Class32-다양한 GAN 모델 활용하기(TBD).ipynb
Class33-Cycle GAN 활용하기.ipynb
Class34-Variational Autoencoder.ipynb
Golden-Retriever (1).jpg
Golden-Retriever (2).jpg
Golden-Retriever (3).jpg
Golden-Retriever (4).jpg
Golden-Retriever (5).jpg
Golden-Retriever-2.jpg
Golden-Retriever.jpg
Golden_Retriever
Koco.ipynb
MNIST/
OLD/
Prob2-1-llc-edl.csv
add.py
best_model.h5
best_model2.h5
cnn.py
cnn2.py
cnn3.py
common/
cvae.gif
data/
deep-learning-from-scratch/
file/
grad_cam.png
image_at_epoch_0000.png
image_at_epoch_0001.png
image_at_epoch_0002.png
image_at_epoch_0003.png
image_at_epoch_0004.png
image_at_epoch_0005.png
image_at_epoch_0006.png
image_at_epoch_0007.png
image_at_epoch_0008.png
image_at_epoch_0009.png
image_at_epoch_0010.png
imagenet-labels (1).txt
imagenet-labels (2).txt
imagenet-labels (3).txt
imagenet-labels (4).txt
imagenet-labels (5).txt

```
imagenet-labels.txt
imagenet_labels.txt
img/
koco/
london_bike_sharing.csv
mmdet_work/
mmdetection/
mnist_L4.hdf5
mnist_test.csv
mnist_train.csv
model.ckpt
model.png
ratings_test.txt
ratings_train.txt
sample_weight.pkl
saved_model/
t10k-images-idx3-ubyte
t10k-images-idx3-ubyte.gz
t10k-labels-idx1-ubyte
t10k-labels-idx1-ubyte.gz
tb_graph/
temp/
train-images-idx3-ubyte
train-images-idx3-ubyte.gz
train-labels-idx1-ubyte
train-labels-idx1-ubyte.gz
train.txt
tuned_models/
valid.txt
과제-test.ipynb
그림.pptx
딥러닝 준비 학습.zip
딥러닝 강의자료.zip
테스트코드/
학생과제/
```

%loadpy를 이용해서 python 파일을 로드 및 실행

In [11]:
```
# %load add.py

def add(x,y):
    return x+y

print(add(4,4))
```

```
8
```

In [12]:
```
# %load add.py

def add(x,y):
    return x+y

print(add(4,4))
```

```
8
```

In [13]:
```
# %load add.py

def add(x,y):
    return x+y

print(add(4,4))
```

```
8
```

%timeit 를 사용해서 실행하는 시간을 점검

In [14]:
```
import numpy as np

x = np.random.random(500)
```

In [15]:
```
%timeit np.sum(x)
```

```
1.67 µs ± 7.21 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)
```

Shell 명령

- Cell에서 shell 명령 가능(!)
  - magic 명령이 같은 부분이 존재
  - os별로 상이함

shell	magic	설명
!pwd, !cd	%pwd, %cd	현재 위치 및 다른 디렉토리로 이동
!env	%env	컴퓨터 환경정보 보기
!echo	%echo	메시지 출력하기
!cp	%cp	카피하기
!ls	%ls	현재 디렉토리의 리스트
!mkdir	%mkdir	디렉토리 생성
!rmdir	%rmdir	디렉토리 삭제
!mv	%mv	파일 이동
!rm	%rm	파일 삭제

In [16]:
```
!dir
```

```
zsh:1: command not found: dir
```